

RAPPORT DE PROJET

# Application Anti-vol

pour téléphone Android

**BINTOUL Giovanni**

BTSSIO 2 —OptionSLAM

Lycée MelkiorGarré

Année scolaire 2024 – 2025

|                                                                                                                                                                                                                       |                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| <b>BTS SERVICES INFORMATIQUES AUX ORGANISATIONS</b><br><b>Épreuve E5 - Conception et développement d'applications (option SLAM)</b><br><b>ANNEXE 7-1-B : Fiche descriptive de réalisation professionnelle (recto)</b> | <b>SESSION 2026</b> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                                                                |                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|---------------------------------|
| <b>DESCRIPTION D'UNE RÉALISATION PROFESSIONNELLE</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                | <b>N° réalisation : 2</b>       |
| <b>Nom, prénom : BINTOUL Giovanni</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |                                                                | <b>N° candidat : 2318021672</b> |
| <b>Épreuve ponctuelle</b> <input checked="" type="checkbox"/>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | <b>Contrôle en cours de formation</b> <input type="checkbox"/> | <b>Date : 08 / 05 / 2026</b>    |
| <b>Organisation support de la réalisation professionnelle</b><br>Lycée Melkior Garré — BTS SIO 2 option SLAM                                                                                                                                                                                                                                                                                                                                                                                                                                  |                                                                |                                 |
| <b>Intitulé de la réalisation professionnelle</b><br>Développement d'une application mobile Android Anti-vol                                                                                                                                                                                                                                                                                                                                                                                                                                  |                                                                |                                 |
| <b>Période de réalisation :</b> Janvier 2025 – Mai 2025 <b>Lieu :</b> Lycée Melkior Garré.....                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                |                                 |
| <b>Modalité :</b> <input checked="" type="checkbox"/> <b>Seul(e)</b> <input type="checkbox"/> <b>En équipe</b>                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                |                                 |
| <b>Compétences travaillées</b><br><input type="checkbox"/> Concevoir et développer une solution applicative<br><input type="checkbox"/> Assurer la maintenance corrective ou évolutive d'une solution applicative<br><input type="checkbox"/> Gérer les données                                                                                                                                                                                                                                                                               |                                                                |                                 |
| <b>Conditions de réalisation<sup>1</sup> (ressources fournies, résultats attendus)</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                                |                                 |
| <b>Description des ressources documentaires, matérielles et logicielles utilisées<sup>2</sup></b><br>Le projet a été réalisé seul, dans le cadre des travaux pratiques de BTS SIO 2 option SLAM.<br>Ressources fournies : PC avec Android Studio, documentation officielle Android, téléphone Android pour les tests.<br>Résultat attendu : une application Android fonctionnelle qui détecte un mouvement brusque et déclenche une alarme sonore arrêtée par code PIN.<br>Logiciels : Android Studio (IDE officiel Google), SDK Android 21+. |                                                                |                                 |
| Langage de programmation : Java.<br>Documentation : developer.android.com (SensorManager, AudioManager, MediaPlayer).<br>Matériel : PC Windows, smartphone Android pour les tests (branchement USB).<br>Ressources complémentaires : Stack Overflow, tutoriels YouTube sur le développement Android.                                                                                                                                                                                                                                          |                                                                |                                 |

<sup>1</sup> En référence aux *conditions de réalisation et ressources nécessaires* du bloc « Conception et développement d'applications » prévues dans le référentiel de certification du BTS SIO.

Les réalisations professionnelles sont élaborées dans un environnement technologique conforme à l'annexe II.E du référentiel du BTS SIO.

Modalités d'accès aux productions<sup>3</sup> et à leur documentation<sup>4</sup>

**BTS SERVICES INFORMATIQUES AUX ORGANISATIONS**

**SESSION 2026**

**Épreuve E5 - Conception et développement d'applications (option SLAM)**

**ANNEXE 7-1-B : Fiche descriptive de réalisation professionnelle  
(verso, éventuellement pages suivantes)**

---

<sup>3</sup>Conformément au référentiel du BTS SIO « Dans tous les cas, les candidats doivent se munir des outils et ressources techniques nécessaires au déroulement de l'épreuve. Ils sont seuls responsables de la disponibilité et de la mise en œuvre de ces outils et ressources. La circulaire nationale d'organisation précise les conditions matérielles de déroulement des interrogations et les pénalités à appliquer aux candidats qui ne se seraient pas munis des éléments nécessaires au déroulement de l'épreuve. ». Les éléments peuvent être un identifiant, un mot de passe, une adresse réticulaire (URL) d'un espace de stockage et de la présentation de l'organisation du stockage.

<sup>4</sup> Lien vers la documentation complète, précisant et décrivant, si cela n'a été fait au verso de la fiche, la réalisation professionnelle, par exemples service fourni par la réalisation, interfaces utilisateurs, description des classes ou de la base de données.

## **Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs**

### **CONTEXTE ET OBJECTIF**

Le projet consiste à développer une application mobile Android dont le but est de protéger un téléphone contre le vol. L'application surveille en permanence les mouvements du téléphone via l'accéléromètre. Dès qu'un mouvement brusque est détecté, une alarme sonore forte se déclenche automatiquement.

### **FONCTIONNALITÉS DÉVELOPPÉES**

1. Activation du mode anti-vol via un bouton (avec saisie d'un code PIN à 4 chiffres).
2. Détection de mouvement via l'accéléromètre : calcul de la magnitude sur les 3 axes (X, Y, Z), seuil fixé à 12 m/s<sup>2</sup>.
3. Déclenchement d'une alarme sonore (fichier MP3 lu en boucle avec MediaPlayer).
4. Mise au maximum automatique du volume via AudioManager (canal STREAM\_ALARM).
5. Désactivation de l'alarme par saisie du code PIN ; restauration du volume initial.

### **TECHNOLOGIES UTILISÉES**

- Android Studio (IDE), Java, SDK Android 21+.
- Classes Android : SensorManager, SensorEventListener, AudioManager, MediaPlayer.
- Interface utilisateur en XML (LinearLayout, EditText, Button, TextView).

### **RÉSULTATS**

L'application est fonctionnelle et stable. Elle a été testée sur un vrai appareil Android. La détection de mouvement est rapide (moins d'une seconde), l'alarme est audible même en mode silencieux, et la désactivation par code PIN fonctionne correctement.

## SOMMAIRE

|                                                 |    |
|-------------------------------------------------|----|
| 1. Introduction                                 | 2  |
| 2. Contexte et besoin identifié                 | 2  |
| 3. Présentation d'Android Studio                | 3  |
| 4. Étude de faisabilité et solutions envisagées | 4  |
| 5. Développement du projet                      | 5  |
| 6. L'interface utilisateur                      | 9  |
| 7. Tests réalisés                               | 10 |
| 8. Difficultés rencontrées                      | 12 |
| 9. Solutions apportées                          | 13 |
| 10. Résultat final                              | 14 |
| 11. Conclusion                                  | 15 |

# 1. Introduction

Dans le cadre de ma deuxième année de BTS SIO option SLAM au Lycée Melkior Garré, j'ai dû réaliser un projet de développement d'application mobile. J'ai choisi de créer une application anti-vol pour téléphone Android.

L'idée m'est venue d'une situation simple : combien de fois pose-t-on son téléphone sur une table dans un café, dans le bus ou à la bibliothèque ? Et combien de fois est-ce qu'on ne fait plus attention à lui pendant quelques minutes ? C'est exactement dans ces moments-là que les vols de téléphone se produisent.

L'objectif de ce projet était donc de créer une application qui surveille le téléphone automatiquement. Quand quelqu'un le touche ou le déplace brusquement, une alarme sonore très forte se déclenche. Elle ne s'arrête que quand on entre le bon code secret.

*Ce rapport présente tout le travail réalisé : pourquoi j'ai choisi ce projet, comment j'ai réfléchi aux différentes solutions, comment l'application a été développée, les problèmes que j'ai rencontrés et ce que j'ai appris.*

## 2. Contexte et besoin identifié

### 2.1 Pourquoi ce projet ?

Le vol de téléphone portable est un problème très fréquent en France. Selon les chiffres qu'on voit régulièrement dans les médias, des milliers de téléphones sont volés chaque année, souvent dans des endroits publics. La plupart du temps, le voleur agit très vite et profite d'un moment d'inattention.

Il existe déjà des solutions, comme les applications de localisation ou les boîtiers avec alarme, mais elles ont souvent des inconvénients : elles demandent une connexion internet, un abonnement payant, ou un accessoire supplémentaire. L'idée ici était de créer quelque chose de simple, directement dans le téléphone, qui fonctionne sans internet.

### 2.2 Les situations à risque

Voici les situations du quotidien qui m'ont inspiré ce projet :

- Poser son téléphone sur une table dans une cafétéria ou un restaurant
- Laisser son téléphone sur le bord du bureau en cours
- Charger son téléphone dans un couloir ou un couloir de transport
- Mettre son téléphone dans son sac ouvert dans le bus ou le métro

Dans toutes ces situations, l'application peut aider à dissuader un voleur ou à alerter les personnes autour.

### 2.3 Ce que l'application doit faire

En réfléchissant au projet, j'ai listé les besoins principaux :

| Besoin                                    | Ce que l'application fait                    |
|-------------------------------------------|----------------------------------------------|
| Détecter un mouvement suspect             | Utilise le capteur de mouvement du téléphone |
| Alerter quand c'est détecté               | Déclenche une alarme sonore forte            |
| Que l'alarme soit vraiment audible        | Met le volume au maximum automatiquement     |
| Que seul le propriétaire puisse l'arrêter | Demande un code PIN à 4 chiffres             |
| Que ce soit simple à utiliser             | Un seul bouton pour activer / désactiver     |

## 3. Présentation d'Android Studio

### 3.1 Qu'est-ce qu'Android Studio ?

Android Studio est le logiciel officiel pour créer des applications Android. C'est ce qu'on appelle un IDE, c'est-à-dire un environnement de développement. En gros, c'est un logiciel tout-en-un qui permet d'écrire le code, de voir l'interface de l'application, de la tester et de la transformer en fichier installable sur un vrai téléphone.

Il est développé par Google et est entièrement gratuit. Il fonctionne sur Windows, Mac et Linux.

### 3.2 Pourquoi Android Studio ?

- C'est l'outil officiel recommandé par Google pour créer des applications Android
- Il est gratuit et bien documenté sur internet
- On peut tester l'application directement sur son téléphone en le branchant en USB
- Il propose un simulateur de téléphone si on n'a pas d'appareil sous la main
- La communauté est grande : on trouve facilement des réponses à ses questions

### 3.3 Le langage utilisé : Java

Pour ce projet, j'ai choisi d'utiliser Java plutôt que Kotlin (l'autre langage compatible avec Android). La raison principale, c'est qu'on apprend Java en cours de SLAM. Ça m'a donc semblé plus logique de travailler avec un langage que je connaissais déjà un peu.

| Java                             | Kotlin                                          |
|----------------------------------|-------------------------------------------------|
| Appris en cours de SLAM          | Plus récent, peu vu en cours                    |
| Beaucoup de ressources en ligne  | Moins de documentation pour débutants           |
| Code plus détaillé, plus lisible | Code plus court mais plus difficile à apprendre |
| Idéal pour débiter sur Android   | Recommandé pour les projets professionnels      |



**Capture d'écran suggérée :** L'interface d'Android Studio ouverte sur le projet — on peut voir le code à gauche et la prévisualisation de l'application à droite.



## 4. Étude de faisabilité et solutions envisagées

### 4.1 Comment détecter le vol ?

La première question à résoudre, c'était : comment l'application sait qu'on est en train de voler le téléphone ? J'ai réfléchi à plusieurs possibilités.

| Solution imaginée      | Comment ça marche                                   | Problème                                   |
|------------------------|-----------------------------------------------------|--------------------------------------------|
| Bouton physique        | L'utilisateur appuie sur un bouton au moment du vol | Ne fonctionne pas si le voleur agit vite   |
| Reconnaissance faciale | L'app reconnaît le visage du propriétaire           | Trop compliqué à développer pour ce niveau |
| Géolocalisation GPS    | L'app détecte si le téléphone change de lieu        | Lent et nécessite internet                 |
| Capteur de mouvement ✓ | Détecte si quelqu'un bouge brusquement le téléphone | Peut avoir de faux déclenchements à régler |

J'ai choisi le capteur de mouvement (l'accéléromètre) parce que c'est la solution la plus naturelle, la plus rapide et la plus simple à mettre en place pour ce type d'application.

### 4.2 Comment alerter en cas de vol ?

Une fois le mouvement détecté, l'application doit alerter. Plusieurs options existent :

- Envoyer un SMS à un contact (nécessite une configuration et internet possible)
- Envoyer une notification (peut être ignorée ou coupée)
- Déclencher une alarme sonore très forte → solution retenue

L'alarme sonore est la meilleure option parce qu'elle attire immédiatement l'attention des personnes autour et elle surprend le voleur. C'est difficile à ignorer.

### 4.3 Récapitulatif des choix techniques

| Ce qu'on voulait faire              | Solution choisie                             |
|-------------------------------------|----------------------------------------------|
| Détecter le vol                     | Accéléromètre (capteur de mouvement intégré) |
| Alerter rapidement                  | Alarme sonore forte en boucle                |
| Que l'alarme soit bien entendue     | Volume automatiquement mis au maximum        |
| Stopper l'alarme de façon sécurisée | Code PIN à 4 chiffres                        |
| Développer l'application            | Android Studio + Java                        |



## 5. Développement du projet

### 5.1 L'accéléromètre : le capteur de mouvement

L'accéléromètre est un petit capteur qui se trouve dans presque tous les téléphones modernes. Son rôle, c'est de mesurer les mouvements du téléphone dans l'espace. C'est lui qui fait que l'écran tourne quand on penche le téléphone, ou que les jeux de course répondent quand on incline le téléphone comme un volant.

Dans notre cas, on l'utilise pour détecter un mouvement brusque, comme quand quelqu'un attrape le téléphone rapidement pour s'enfuir.

*Pour faire simple : imaginez que le téléphone est comme une bulle à niveau. Quand il est immobile, la bulle est au centre. Dès que quelqu'un le bouge brusquement, la bulle se déplace fortement. C'est ce déplacement que l'accéléromètre mesure.*

### 5.2 Comment le capteur détecte un mouvement suspect ?

L'accéléromètre envoie en permanence des données sur le mouvement du téléphone. Il donne trois valeurs : X (gauche/droite), Y (haut/bas) et Z (avant/arrière). Pour savoir si le mouvement est vraiment brusque, on combine ces trois valeurs en une seule valeur globale.

On compare ensuite cette valeur globale à un seuil qu'on a fixé. Si la valeur dépasse ce seuil, on considère que le mouvement est suspect et l'alarme se déclenche.

| Situation                       | Valeur de mouvement approximative |
|---------------------------------|-----------------------------------|
| Téléphone posé, immobile        | Proche de 0                       |
| Légère vibration (notification) | Entre 1 et 3                      |
| Pris en main normalement        | Entre 3 et 7                      |
| Mouvement brusque / vol simulé  | Au-dessus de 10                   |
| Notre seuil de déclenchement    | 12 — bien au-dessus du quotidien  |

On a choisi un seuil de 12 après plusieurs tests. Cette valeur permet d'éviter les faux déclenchements tout en restant réactif en cas de vrai vol.



**Capture d'écran suggérée :** L'écran de l'application avec la valeur d'accélération affichée en bas, pendant un test de mouvement.

### 5.3 Comment le capteur est utilisé dans le code ?

Pour utiliser le capteur dans l'application, on passe par trois étapes principales :

**1**

### **Accéder au capteur**

On demande à Android d'accéder à l'accéléromètre. C'est comme brancher un appareil : on utilise un gestionnaire de capteurs fourni par Android pour le connecter.

**2**

### **Écouter le capteur**

On dit à l'application de surveiller en permanence ce que le capteur envoie. À chaque nouvelle mesure (environ 5 fois par seconde), une fonction de l'application est appelée automatiquement avec les nouvelles valeurs.

**3**

### **Analyser et réagir**

Dans cette fonction, on calcule si le mouvement dépasse notre seuil. Si oui, on déclenche l'alarme. Sinon, on attend la prochaine mesure.

Quand on désactive la protection, on dit à Android d'arrêter d'envoyer les données du capteur. Ça permet d'économiser la batterie.

## **5.4 Le déclenchement de l'alarme**

Quand un mouvement suspect est détecté, deux choses se passent exactement en même temps :

- Le volume du téléphone est mis au maximum
- Le son d'alarme démarre en boucle

Pour le son, j'ai utilisé un fichier audio (un son de sirène forte) que j'ai placé directement dans les ressources de l'application. Android propose une façon simple de jouer des sons avec une option "boucle" : quand le son se termine, il repart automatiquement depuis le début. L'alarme tourne donc en continu jusqu'à ce qu'on entre le bon code.

*Exemple concret : quelqu'un prend le téléphone sur une table → moins d'une seconde plus tard, une alarme forte retentit. Les personnes autour se retournent, le voleur est surpris. Dans la majorité des cas, il repose le téléphone.*

## **5.5 La gestion du volume**

Une alarme qui sonne en mode silencieux, ça ne sert à rien. C'est pour ça que l'application modifie le volume automatiquement au moment du déclenchement.

Android propose plusieurs "canaux" de volume : un pour la musique, un pour les appels, un pour les notifications... et un spécialement pour les alarmes. On utilise ce canal "alarme" parce qu'il est le seul qui fonctionne même quand le téléphone est en mode silencieux. C'est exactement ce dont on a besoin.

Voici ce que l'application fait en matière de volume :

1

#### **Sauvegarde le volume actuel**

Avant de changer quoi que ce soit, on note le volume qu'avait le téléphone. Ça permettra de le remettre à l'identique après.

2

#### **Met le volume au maximum**

On demande à Android d'augmenter le volume du canal alarme à son maximum possible.

3

#### **Restaure le volume après**

Quand l'alarme est désactivée avec le code PIN, le volume revient automatiquement à ce qu'il était avant.



**Capture d'écran suggérée :** L'indicateur de volume du téléphone au maximum, déclenché automatiquement par l'application au moment de l'alarme.

## **5.6 Le système de code PIN**

Le code PIN, c'est ce qui permet uniquement au propriétaire du téléphone d'arrêter l'alarme. Sans ça, n'importe qui pourrait couper l'alarme.

Le fonctionnement est simple :

- Avant d'activer la protection, l'utilisateur choisit un code à 4 chiffres
- Ce code est retenu par l'application pendant toute la durée de la protection
- Quand l'alarme se déclenche, un champ de saisie apparaît à l'écran
- Si on entre le bon code → l'alarme s'arrête et tout revient à la normale
- Si le code est mauvais → un message d'erreur s'affiche et l'alarme continue

*Important : le code PIN n'est pas sauvegardé sur le téléphone de façon permanente dans cette version. Si on ferme l'application, le code est perdu. C'est une amélioration qu'on peut envisager plus tard.*

Pour vérifier le code, on compare simplement ce que l'utilisateur a tapé avec ce qu'il avait saisi au départ. Si c'est identique, l'alarme s'arrête. C'est une vérification très basique mais suffisante pour ce projet.

## 6. L'interface utilisateur

### 6.1 Conception de l'écran principal

Pour l'interface, j'ai voulu rester le plus simple possible. L'application n'a qu'un seul écran. L'idée, c'est que l'utilisateur puisse activer la protection en quelques secondes, sans avoir à chercher ou naviguer entre plusieurs pages.

L'écran est divisé en plusieurs zones :

- En haut : le titre de l'application et un indicateur de statut (vert = protection active, rouge = inactive)
- Au milieu : un champ pour saisir le code PIN et le bouton d'activation
- En bas : une zone qui apparaît uniquement quand l'alarme se déclenche, avec le champ pour entrer le code et arrêter l'alarme
- Tout en bas : un petit affichage de la valeur de mouvement, utile pour les tests

### 6.2 Les couleurs et les indicateurs visuels

J'ai utilisé un code couleur simple pour que l'utilisateur comprenne l'état de l'application d'un seul coup d'œil :

| Couleur                  | Signification                             |
|--------------------------|-------------------------------------------|
| Vert (bouton et statut)  | Protection inactive — on peut activer     |
| Rouge (bouton et statut) | Protection active — surveillance en cours |
| Fond rose / rouge        | Alarme en cours — entrer le code PIN      |



**Capture d'écran suggérée :** L'écran principal avec le bouton vert 'ACTIVER LA PROTECTION' et le champ de code PIN visible.



**Capture d'écran suggérée :** L'écran quand l'alarme est déclenchée : fond rouge, champ de saisie du code PIN visible en bas.

### 6.3 Ce qui change à l'écran selon l'état

| État de l'application        | Ce qu'on voit à l'écran                     | Ce qu'on peut faire       |
|------------------------------|---------------------------------------------|---------------------------|
| Protection inactive (départ) | Bouton vert, statut rouge, champ PIN activé | Entrer le code et activer |
| Protection active            | Bouton rouge, statut vert, champ PIN grisé  | Désactiver ou attendre    |

|                   |                                           |                             |
|-------------------|-------------------------------------------|-----------------------------|
| Alarme déclenchée | Zone rouge visible, champ<br>code affiché | Entrer le code pour stopper |
|-------------------|-------------------------------------------|-----------------------------|

## 7. Tests réalisés

### 7.1 Comment j'ai testé l'application

J'ai testé l'application directement sur mon téléphone Android. Pour ça, j'ai branché le téléphone à mon PC avec un câble USB et j'ai lancé l'application depuis Android Studio. Le téléphone doit avoir le "mode développeur" activé pour accepter l'installation depuis Android Studio.

J'ai réalisé plusieurs types de tests pour vérifier que tout fonctionnait correctement.

### 7.2 Scénarios de test

| Scénario testé              | Ce que j'ai fait concrètement                 | Résultat                                 |
|-----------------------------|-----------------------------------------------|------------------------------------------|
| Test 1 – Code PIN invalide  | Laisser le champ vide et appuyer sur Activer  | Message d'erreur affiché ✓               |
| Test 2 – Activation normale | Saisir 1234 et appuyer sur Activer            | Bouton rouge, surveillance active ✓      |
| Test 3 – Pas de mouvement   | Poser le téléphone sur une table et attendre  | Aucune alarme ✓                          |
| Test 4 – Mouvement léger    | Faire vibrer doucement la table               | Aucune alarme (seuil non atteint) ✓      |
| Test 5 – Mouvement brusque  | Saisir rapidement le téléphone et le soulever | Alarme déclenchée en moins d'1 seconde ✓ |
| Test 6 – Mauvais code PIN   | Entrer un code incorrect (ex : 9999)          | Message erreur, alarme continue ✓        |
| Test 7 – Bon code PIN       | Entrer le bon code (1234)                     | Alarme stoppée, volume restauré ✓        |
| Test 8 – Désactivation      | Appuyer sur 'Désactiver la protection'        | Retour à l'état initial ✓                |

### 7.3 Tests de fiabilité du capteur

J'ai aussi réalisé des tests spécifiques pour calibrer le seuil de détection. L'objectif était de trouver une valeur qui ne déclenche pas l'alarme pour les mouvements normaux, mais qui la déclenche immédiatement pour un mouvement de vol.

| Mouvement testé                    | Alarme déclenchée ? |
|------------------------------------|---------------------|
| Téléphone posé immobile            | Non ✓               |
| Notification reçue (vibration)     | Non ✓               |
| Téléphone pris doucement en main   | Non ✓               |
| Téléphone posé brusquement sur une | Non ✓               |



|                              |       |
|------------------------------|-------|
| table                        |       |
| Téléphone arraché rapidement | Oui ✓ |
| Téléphone secoué fortement   | Oui ✓ |

Ces tests m'ont confirmé que le seuil de 12 est une bonne valeur pour ce projet. Il y a encore quelques cas limites (comme frapper fort la table), mais dans les situations réelles de vol, l'alarme se déclenche bien.

## 8. Difficultés rencontrées

### 8.1 Trouver le bon seuil de sensibilité

---

C'est la difficulté principale que j'ai rencontrée. Au départ, j'avais mis un seuil trop bas. L'alarme partait pour n'importe quel mouvement : poser le téléphone un peu fort sur une table, marcher avec dans la poche, ou même juste soulever le téléphone normalement. C'était complètement inutilisable.

À l'inverse, quand j'avais mis un seuil trop élevé, l'application ne réagissait plus du tout. Même en secouant le téléphone fort, rien ne se passait. J'ai dû faire beaucoup de tests pour trouver une valeur équilibrée.

### 8.2 L'alarme qui plantait l'application

---

Audébut, l'application se fermait parfois toute seule quand on essayait d'arrêter l'alarme. Après quelques recherches, j'ai compris pourquoi : quand on joue un son avec Android, il faut libérer les ressources qu'il utilise après l'arrêt. Si on ne le fait pas, des erreurs peuvent apparaître au lancement suivant.

J'ai mis du temps à trouver ce problème parce que l'erreur ne s'affichait pas toujours au même moment. C'est en regardant les journaux d'erreur d'Android Studio que j'ai fini par comprendre.

### 8.3 Les faux déclenchements

---

Même avec le bon seuil, il arrivait parfois que l'alarme parte sans raison apparente. Après observation, j'ai réalisé que certaines vibrations du téléphone (une notification forte, ou poser le téléphone sur une surface qui vibre légèrement) pouvaient parfois suffire à dépasser le seuil.

### 8.4 Comprendre le cycle de vie d'une application Android

---

Android gère les applications d'une façon particulière. Quand on quitte une application (par exemple parce qu'on reçoit un appel), elle ne se ferme pas vraiment : elle se met en pause. Quand on y revient, elle reprend. Au début, je ne savais pas comment gérer ça, et le capteur continuait de fonctionner même quand l'application était en arrière-plan, ce qui consommait de la batterie.

### 8.5 Mettre en place le bouton pour modifier le volume

---

Je pensais que modifier le volume serait simple, mais Android a plusieurs "niveaux" de volume différents selon le contexte (musique, appels, alarmes, etc.). Il a fallu que je comprenne lequel utiliser pour que l'alarme soit audible même si le téléphone est en mode silencieux.

## 9. Solutions apportées

### 9.1 Le seuil de sensibilité

Après de nombreux tests avec différentes valeurs, j'ai fixé le seuil à 12. J'ai aussi ajouté l'affichage de la valeur de mouvement en temps réel en bas de l'écran (un petit texte discret). Ça m'a permis de voir exactement quelles valeurs correspondaient à quel type de mouvement, et de régler précisément le seuil.

### 9.2 La gestion correcte du son

Pour corriger les problèmes de son, j'ai ajouté deux choses importantes. D'abord, une vérification systématique : avant d'essayer d'arrêter le son, on vérifie qu'il est bien en train de jouer. Ensuite, on libère correctement les ressources après chaque utilisation. Ces deux ajouts ont suffi à rendre l'application stable.

### 9.3 Réduction des faux déclenchements

Pour réduire les faux déclenchements, j'ai légèrement augmenté le seuil et j'ai choisi une fréquence de lecture du capteur moins rapide. Au lieu de lire les données 60 fois par seconde, on les lit environ 5 fois par seconde. Ça suffit largement pour détecter un vol, et ça évite de réagir à des micro-vibrations passagères.

### 9.4 La gestion du cycle de vie

Une fois que j'ai compris comment Android fonctionne, j'ai ajouté deux méthodes dans le code : une pour mettre le capteur en pause quand l'application passe en arrière-plan, et une pour le relancer quand elle revient au premier plan. Résultat : plus de surconsommation de batterie, et l'application se comporte normalement.

### 9.5 Le bon canal de volume

Après quelques recherches, j'ai trouvé que le canal "Alarm" d'Android est le bon à utiliser. C'est le seul canal qui fonctionne même en mode silencieux, ce qui est exactement ce dont on a besoin pour une application anti-vol. Une fois que j'ai changé de canal, le volume fonctionnait parfaitement.

## 10. Résultat final

### 10.1 Ce que fait l'application aujourd'hui

L'application est aujourd'hui fonctionnelle et stable. Voici exactement ce qui se passe quand on l'utilise :

1

#### Démarrage

L'utilisateur ouvre l'application. Il voit un écran simple avec un champ pour entrer son code PIN et un gros bouton vert.

2

#### Activation

L'utilisateur tape son code à 4 chiffres et appuie sur le bouton. Le bouton passe au rouge, le statut passe au vert : la surveillance est active.

3

#### Surveillance

Le téléphone surveille silencieusement les mouvements. L'utilisateur peut poser le téléphone et vaquer à autre chose.

4

#### Détection

Quelqu'un touche le téléphone brusquement. En moins d'une seconde, l'alarme se déclenche : volume au maximum, son de sirène en boucle.

5

#### Désactivation

Le propriétaire entre son code PIN. L'alarme s'arrête, le volume revient à son niveau normal, l'application repasse en mode de surveillance.



**Capture d'écran suggérée :** L'application en mode surveillance actif — bouton rouge, statut vert, champ PIN désactivé.

### 10.2 Bilan de l'application

| Ce qui fonctionne bien                                                                                                                                                                                                                                                        | Ce qui pourrait être amélioré                                                                                                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>✓ Détection rapide du mouvement</li><li>✓ Alarme forte et en boucle</li><li>✓ Volume au maximum automatique</li><li>✓ Désactivation par code PIN</li><li>✓ Interface simple, un seul écran</li><li>✓ Fonctionne sans internet</li></ul> | <ul style="list-style-type: none"><li>✗ Code PIN non sauvegardé à la fermeture</li><li>✗ L'app doit rester ouverte pour fonctionner</li><li>✗ Pas d'envoi de message automatique</li><li>✗ Pas de personnalisation du seuil par l'utilisateur</li></ul> |

## 11. Conclusion

### Ce que j'ai appris

---

Ceprojetm'a beaucoup appris, autant sur le plan technique que personnel. C'est le premier projet Android que je développe vraiment de A à Z, et c'était une expérience très enrichissante.

Avant de commencer, je ne savais pas du tout comment accéder aux capteurs d'un téléphone, ni comment gérer le son depuis une application. J'ai dû apprendre à utiliser des outils fournis par Android que je ne connaissais pas. Parfois, j'ai aussi compris que le développement mobile, c'est beaucoup de tests et de petits ajustements.

Ce qui m'a le plus appris, c'est probablement la gestion des erreurs et le débogage. Quand l'application plantait, il fallait analyser les messages d'erreur dans Android Studio pour comprendre ce qui se passait. C'est quelque chose qu'on ne fait pas vraiment en cours, et c'est très utile dans la vraie vie.

### Les compétences développées

---

- Utiliser Android Studio pour créer, tester et déployer une application
- Accéder aux capteurs d'un téléphone (l'accéléromètre) depuis le code
- Gérer le son et le volume d'un téléphone Android
- Créer une interface simple et claire avec des éléments qui changent selon l'état
- Comprendre le cycle de vie d'une application Android (pause, reprise, fermeture)
- Tester une application sur un vrai appareil et corriger les bugs
- Lire des messages d'erreur et chercher des solutions sur la documentation Android

### Les améliorations possibles

---

Si ce projet devait continuer, voici ce que j'aimerais ajouter en priorité :

- Sauvegarder le code PIN de façon permanente, pour ne pas avoir à le remettre à chaque ouverture de l'application
- Faire fonctionner la surveillance en arrière-plan, même quand l'application n'est pas à l'écran
- Ajouter un délai de quelques secondes entre l'activation et le début de la surveillance, pour laisser le temps de poser le téléphone
- Permettre à l'utilisateur de régler lui-même la sensibilité selon ses besoins
- Envoyer un SMS automatique à un contact de confiance quand l'alarme se déclenche

*Ce projet m'a montré qu'avec des connaissances de base en programmation, on peut créer des applications utiles et concrètes. Le plus important, c'est de bien réfléchir à ce qu'on veut faire avant de coder, et d'avancer étape par étape.*

## Pour finir

---

Jesuis plutôt satisfait du résultat pour un premier projet Android. L'application fait ce qu'elle est censée faire : elle détecte un mouvement brusque, déclenche une alarme forte et ne s'arrête qu'avec le bon code. C'est simple, mais c'est efficace.

Ce type de projet m'a aussi confirmé que le développement mobile est quelque chose qui m'intéresse et que j'aimerais continuer à approfondir après mon BTS.